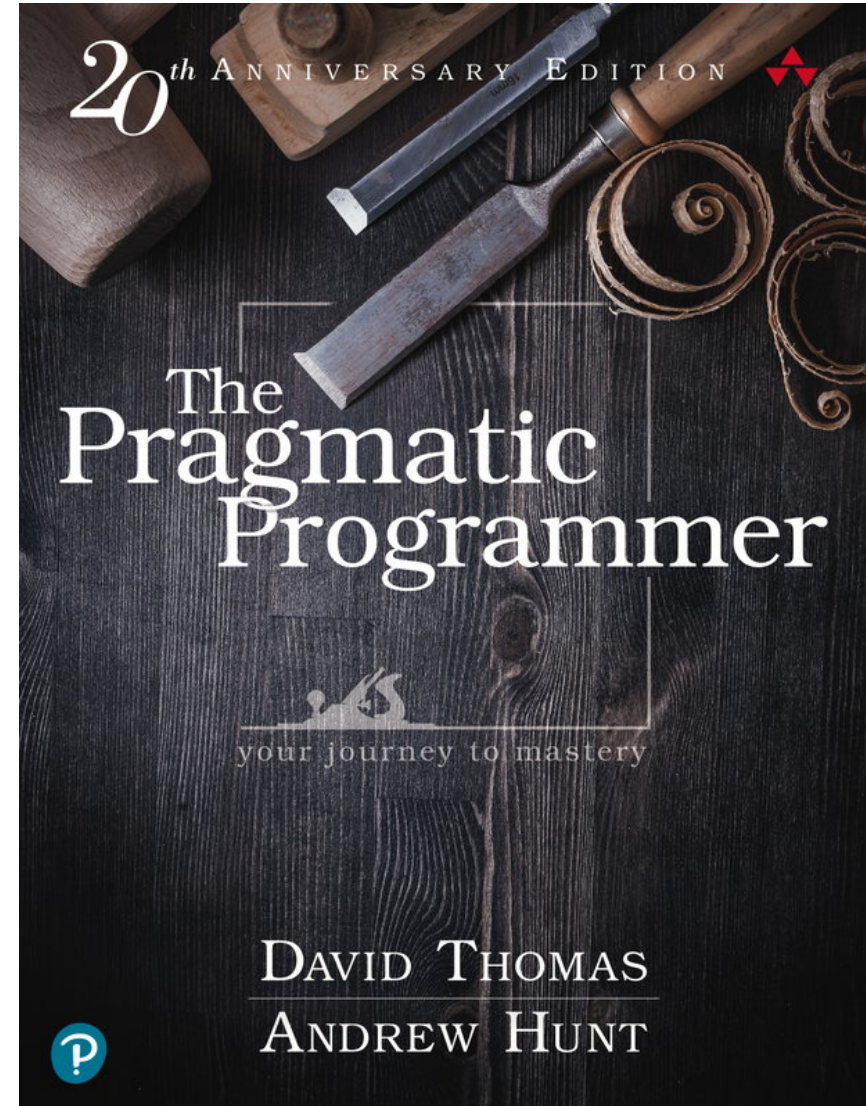
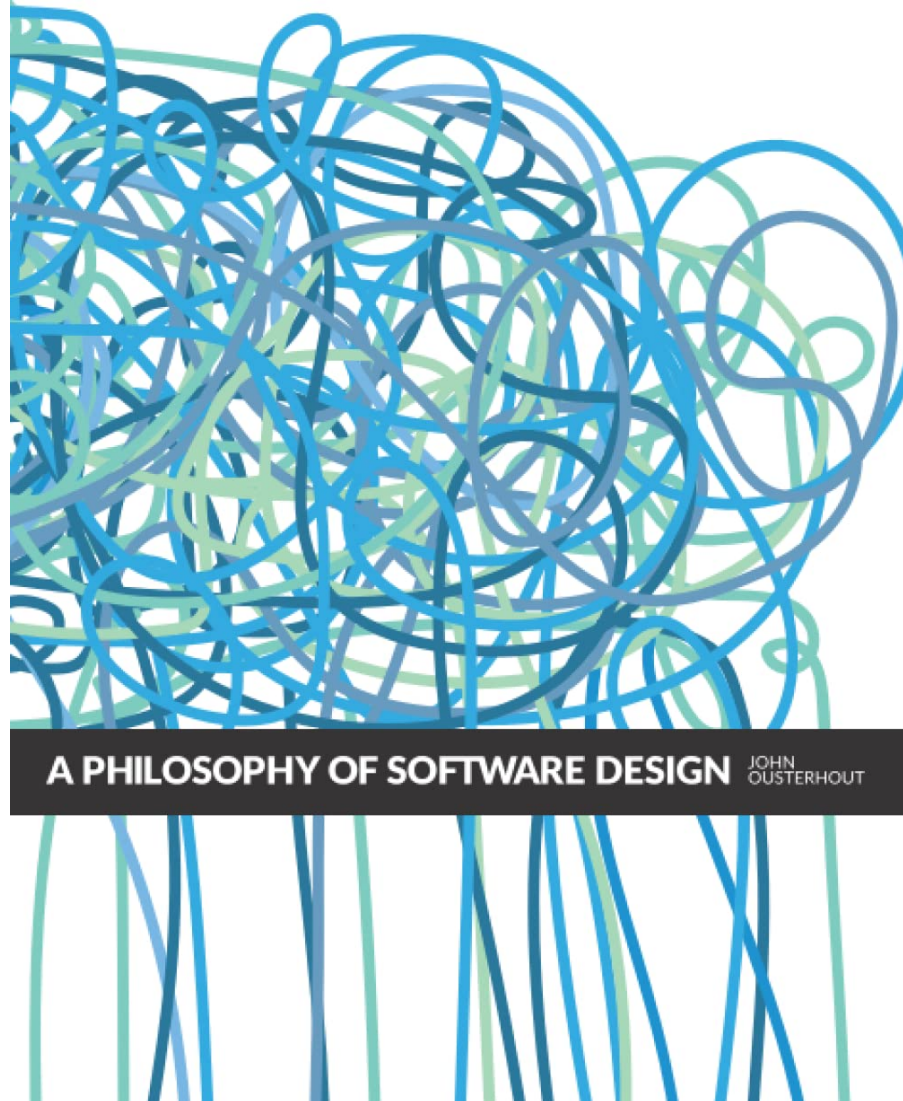


Software Engineering

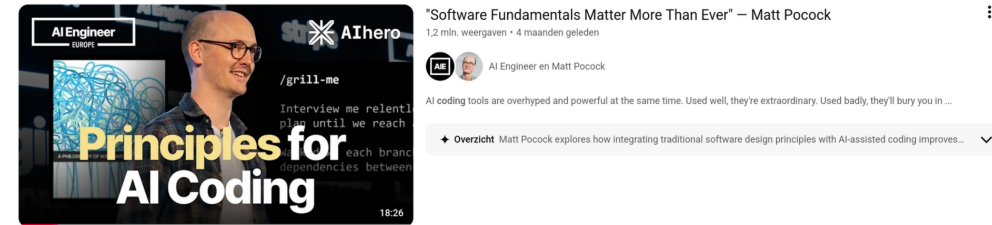
Saeyslab Conference 2026

Sources

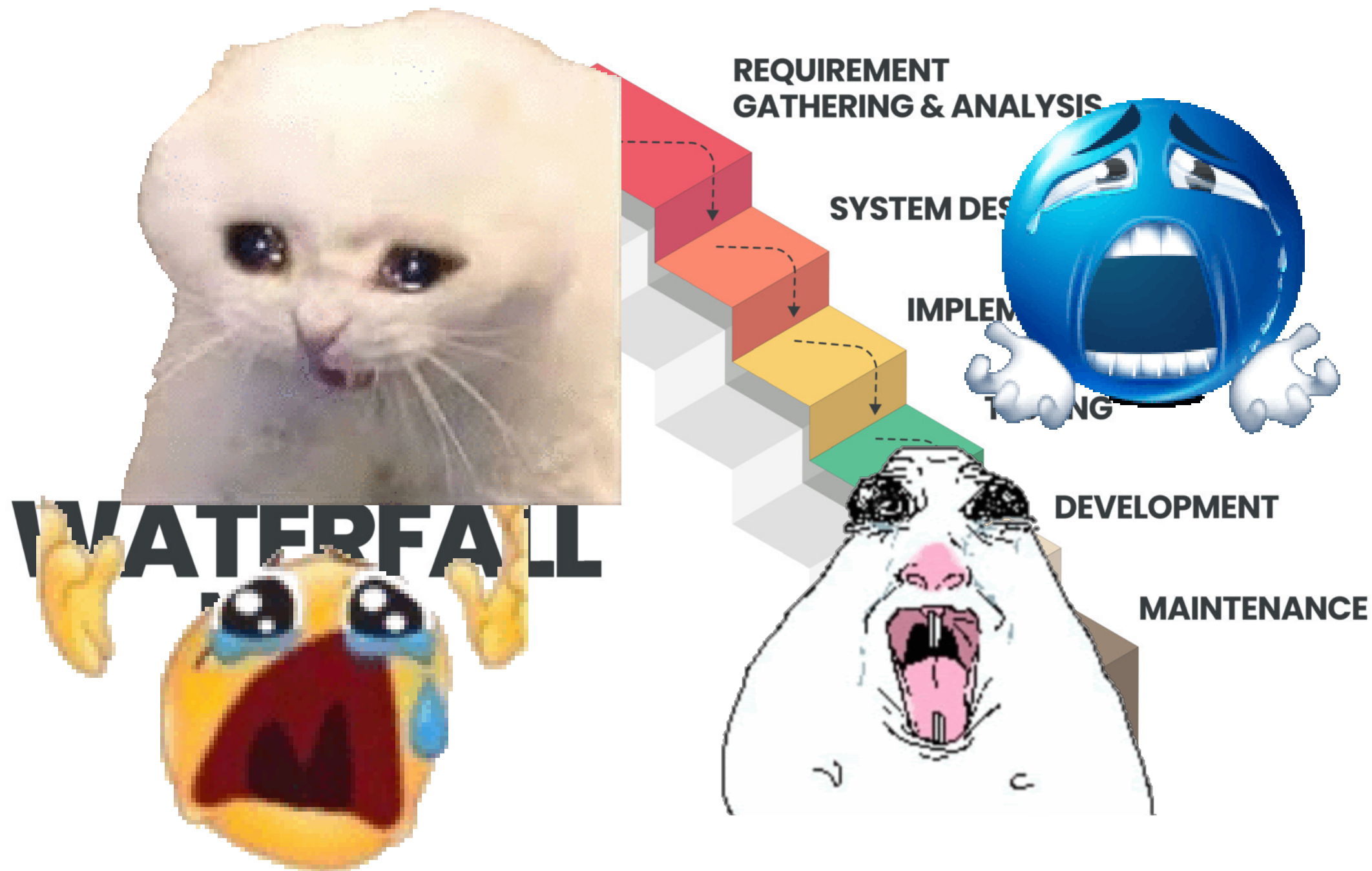


Why do we need “good code”?

- Improves collaboration
 - ... with others
 - ... with your future self
 - ... with coding agents
- Improves reproducibility
- Makes it easier to discover and fix bugs
- Makes it easier to make changes
 - Which you will be doing at revision time



<https://ai.engineer/worldsfair/2026>



Complexity

Complexity is anything related to the structure of a software system that makes it hard to understand and modify the system.

Note: this is not the same as “has lots of features”

Symptoms of complexity

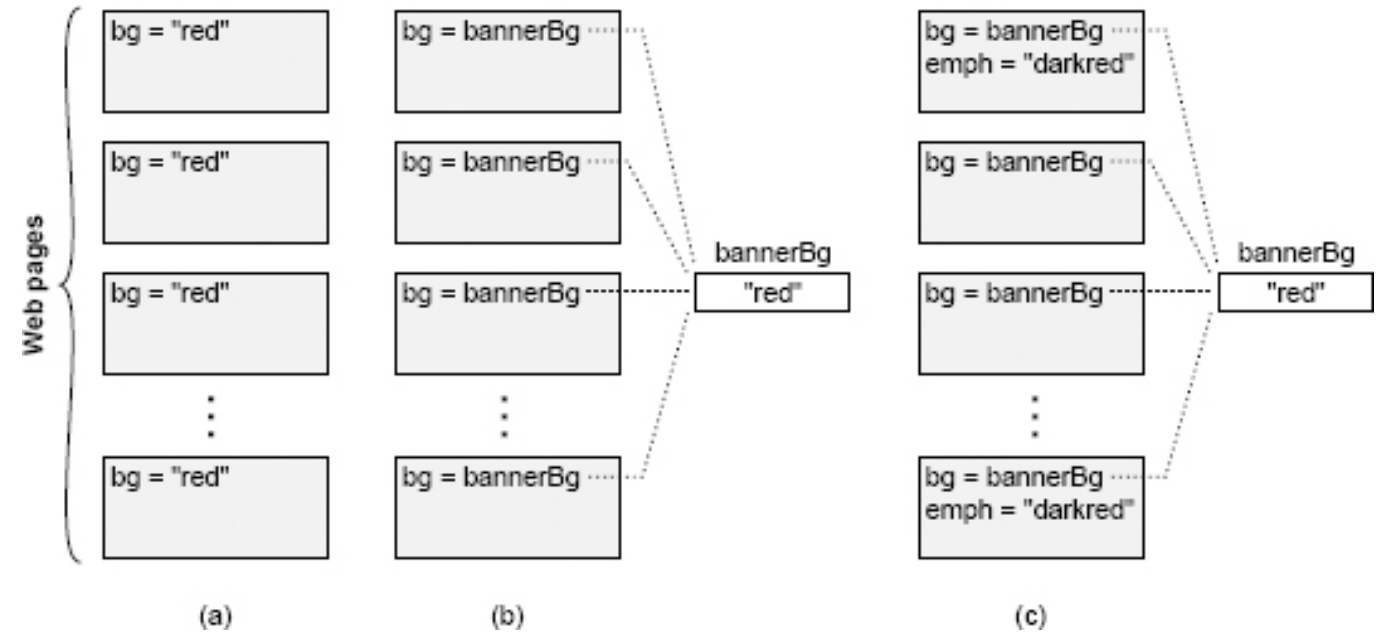
- Change amplification
- Cognitive load
- Unknown unknowns

Causes of complexity

- Dependencies
- Obscurity

Complexity: example

- Change amplification
- Cognitive load
- Unknown unknowns
- Dependencies
- Obscurity



สมาคมการแพทย์
2258641, 2224079

SIAM GREEN
CANNABIS
BANGKOK

325

Properties of good code

- Good code is obvious
- Good code is easy to change (ETC)
- Good code doesn't repeat itself (DRY)

Good code is obvious

```
raw = {}  
ds = load(fp)  
for n, X in ds.items():  
    for mtr in fns:  
        raw[n] = mtr(X)  
res = Result(raw)  
res.write(fpo)
```

Good code is obvious

```
raw_results = {}  
datasets_dict = load_datasets(file_path)  
for dataset_name, dataset_numpy in datasets_dict.items():  
    for metric_fn in metric_fns:  
        raw_results[dataset_name] = metric_fn(dataset_numpy)  
benchmark_result = BenchmarkResult(raw_results)  
benchmark_result.write(output_file_path)
```

Working Code Is Not Enough

- **Tactical programming:** get it working
 - *Programming by coincidence*
- **Strategic programming:** investment mindset
 - *Deliberate programming*

Reducing complexity

- Make code **simpler** and more **obvious**
- Make code more **modular**

Modular Design

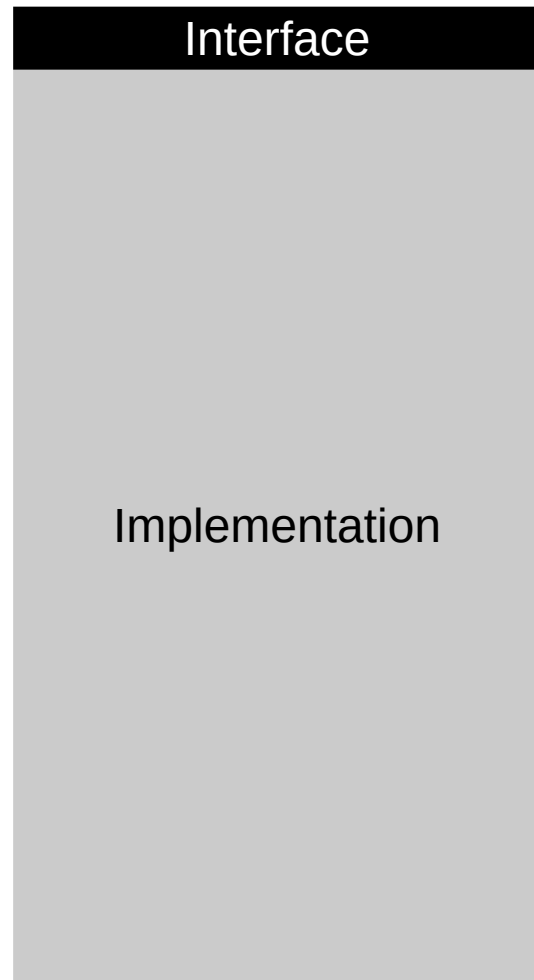
Modular Design

- A system is decomposed into independent **modules**
- A module has an **interface** and an **implementation**
 - Interface: what a developer needs to know to **use** the module
 - Implementation: code that carries out promises made by the interface
- The interface is **what**, the implementation is **how**

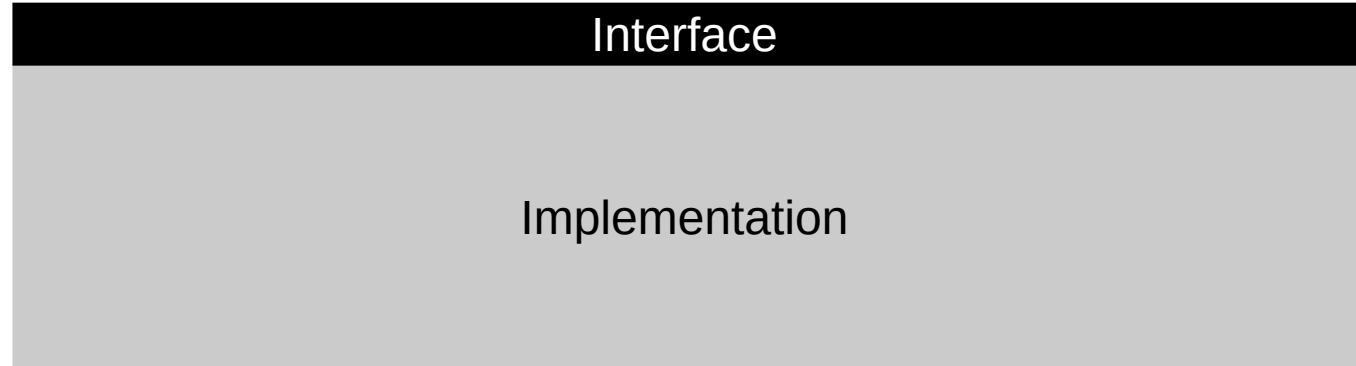
Abstraction

- An *abstraction* is a simplified view of an entity, which **omits unimportant details**.
- An abstraction that omits **important details** is a *false abstraction*.
- The key to designing abstractions:
 - Understand what is important
 - Minimize the amount of information that is important

Modules should be deep



Deep module



Shallow module

Example of a deep module

```
// Open a file, given a path and optional extra flags
int open(const char* path, int flags, mode_t permissions);

// Read a set number of bytes from the file
ssize_t read(int fd, void* buffer, size_t count);

// Write a given text to the file
ssize_t write(int fd, const void* buffer, size_t count);

// Go to a specific position in the file
off_t lseek(int fd, off_t offset, int referencePosition);

// Close the file
int close(int fd);
```

Example of a shallow method

```
def setAttributeToNone(attribute: str):  
    data.set(attribute, None)
```

Red Flag: Shallow Module

- A shallow module is one whose interface is complicated relative to the functionality it provides.
- Shallow modules don't help much in the battle against complexity, because the benefit they provide (not having to learn about how they work internally) is negated by the cost of learning and using their interfaces.
- Small modules tend to be shallow.

Information Hiding and Leakage

- Deep modules **hide information** from their users
- The opposite is **information leakage**:
a design decision is reflected in multiple modules

Example of Information Leakage

```
reader = FileReader()  
data = reader.read(path)  
processed_data = process(data)  
writer = FileWriter()  
writer.write(path)
```

Red Flag: Information Leakage

Information leakage occurs when the same knowledge is used in multiple places, such as two different classes that both understand the format of a particular type of file.

Red Flag: Temporal Decomposition

*In temporal decomposition,
execution order is reflected in the code structure:
operations that happen at different times
are in different methods or classes.*

*If the same knowledge is used at different points in execution,
it gets encoded in multiple places, resulting in information leakage*

Classitis

```
FileInputStream fileStream = new FileInputStream(fileName);  
BufferedInputStream bufferedStream = new BufferedInputStream(fileStream);  
ObjectInputStream objectStream = new ObjectInputStream(bufferedStream);
```

Red Flag: Overexposure

If the API for a commonly used feature forces users to learn about other features that are rarely used, this increases the cognitive load on users who don't need the rarely used features.

General-Purpose Modules are Deeper

- A more general-purpose module can find unanticipated uses in the future (see also: strategic programming, investment mindset)
- However, it's hard to predict possible future uses
- Rule of thumb: make modules **somewhat general-purpose**: the *functionality* should reflect your current needs, but the *interface* should not.

General-Purpose Modules are Deeper: Example

- **Task:**
 - Build a simple GUI text editor
 - Support point, click, type to edit, undo, redo

Special-purpose approach:

```
class Text:
    def backspace(self, cursor: Cursor): ...
    def delete(self, cursor: Cursor): ...
    def delete_selection(self, selection: Selection): ...
    def insert(self, cursor: Cursor, text: str): ...

class GUI:
    def backspace(self):
        self.text.backspace(self.cursor)

    def delete(self):
        self.text.delete(self.cursor)
```

General-purpose approach:

```
class Text:
    def insert(position: Position, text: str): ...
    def delete(start: Position, end: Position): ...
    def change_position(position: Position, num_chars: int): ...

class GUI:
    def backspace(self):
        self.text.delete(self.text.change_position(self.cursor, -1), self.cursor)

    def delete(self):
        self.text.delete(self.cursor, self.text.change_position(self.cursor, 1))
```

Question: What if you had to add find-and-replace?

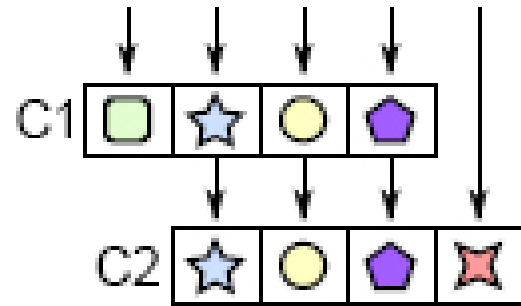
Questions to ask yourself:

- What is the simplest interface that will cover all of my current needs?
- In how many situations will this method be used?
- Is this interface easy to use for my current needs?

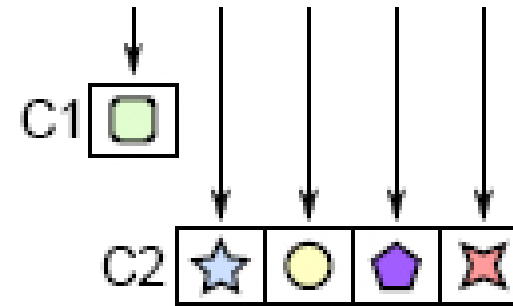
Red Flag: Pass-through methods

A pass-through method is one that does nothing except pass its arguments to another method, usually with the same API as the pass-through method. This typically indicates that there is not a clean division of responsibility between the classes.

Fixing pass-through methods



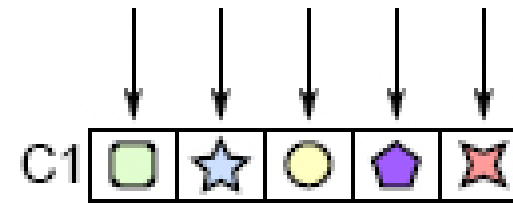
(a)



(b)



(c)



(d)

Inheritance

- Interface inheritance
- Implementation inheritance

Implementation inheritance

```
class Vehicle:
    def __init__(self, speed):
        self.speed = speed

    def stop(self):
        self.speed = 0

    def move_at(self, speed):
        self.speed = speed

class Car(Vehicle):
    def info(self):
        print(f"I'm driving at {self.speed}")

car = Car(0)
car.move_at(30)
car.info()
```

Interface inheritance

```
class Drivable:
    def get_speed(self):
        raise NotImplementedError()

    def stop(self):
        raise NotImplementedError()

class Locatable:
    def get_location(self) -> Coordinate:
        raise NotImplementedError()

class Car(Drivable, Locatable):
    ...
```

Red Flag: implementation inheritance

Implementation inheritance creates dependencies between the parent class and each of its subclasses. Class instance variables in the parent class are often accessed by both the parent and child classes; this results in information leakage between the classes in the inheritance hierarchy and makes it hard to modify one class in the hierarchy without looking at the others.

Comments

Writing good comments: the four excuses

- ***Good code is self-documenting***
 - If users must read the code of module to use it, then there is no abstraction
- ***I don't have time to write comments***
 - Writing good comments will save you time in the long run
- ***Comments get out of date and become misleading***
 - This can be avoided, see later
- ***All the comments I have seen are worthless***
 - Might be true! But this problem is solvable

Benefits of good comments

- Symptoms of complexity:
 - **Change amplification:** a simple change requires modifications in many places
 - **Cognitive load:** a developer must know a lot of information to make a change
 - **Unknown unknowns:** it is unclear what code needs to be modified to make a change
- Good comments mitigate **cognitive load** and **unknown unknowns**
- Causes of complexity:
 - **Dependencies:** documentation can clarify dependencies
 - **Obscurity:** documentation can eliminate obscurity

```
# Get pointer copy
ptr_copy = get_copy(obj)

# Is obj free?
if is_unlocked(ptr_copy):
    # return current obj
    return obj

# Already a copy?
if is_copy(ptr_copy):
    # return obj
    return obj

thread_id = get_thread_id(ptr_copy)
# Locked by current ctx
if thread_id == ctx.thread_id:
    # Return copy
    return ptr_copy
```

Red Flag: Comment repeats code

If the information in a comment is already obvious from the code next to the comment, then the comment isn't helpful.

One example of this is when the comment uses the same words that make up the name of the thing it is describing.

Tip: use different words in the comment

```
# Horizontal padding of each line in the text  
padding = 4  
  
# The amount of blank space to leave  
# on the left and right size of each line in the text,  
# in pixels  
padding = 4
```

Lower-level comments add precision

```
# Current offset in resp Buffer
offset: int = 4

# Contains all line-widths inside the document and
# number of appearances.
line_widths: dict[int, int] = {}

# Position in this buffer of the first object that hasn't
# been returned to the client
offset: int = 4

# Holds statistics about line lengths of the form <length, count>
# where length is the number of characters in a line (including
# the newline), and count is the number of lines with
# exactly that many characters. If there are no lines with
# a particular length, then there is no entry for that length.
line_widths: dict[int, int] = {}
```

Higher-level comments add intuition

```
// If there is a LOADING readRpc using the same session
// as PKHash pointed to by assignPos, and the last PKHash
// in that readRPC is smaller than current assigning
// PKHash, then we put assigning PKHash into that readRPC.
int readActiveRpcId = RPC_ID_NOT_ASSIGNED;
for (int i = 0; i < NUM_READ_RPC; i++) {
    if (session == readRpc[i].session
        && readRpc[i].status == LOADING
        && readRpc[i].maxPos < assignPos
        && readRpc[i].numHashes < MAX_PKHASHES_PERRPC) {
        readActiveRpcId = i;
        break;
    }
}
```

Higher-level comments add intuition

```
// Try to append the current key hash onto an existing
// RPC to the desired server that hasn't been sent yet.
int readActiveRpcId = RPC_ID_NOT_ASSIGNED;
for (int i = 0; i < NUM_READ_RPC; i++) {
    if (session == readRpc[i].session
        && readRpc[i].status == LOADING
        && readRpc[i].maxPos < assignPos
        && readRpc[i].numHashes < MAX_PKHASHES_PERRPC) {
        readActiveRpcId = i;
        break;
    }
}
```

Interface documentation

This class implements the client side framework for index range lookups. It manages a single LookupIndexKeys RPC and multiple IndexedRead RPCs. Client side just includes "IndexLookup.h" in its header to use IndexLookup class. Several parameters can be set in the config below:

- The number of concurrent indexedRead RPCs
- The max number of PKHashes a indexedRead RPC can hold at a time
- The size of the active PKHashes

To use IndexLookup, the client creates an object of this class by providing all necessary information. After construction of IndexLookup, client can call getNext() function to move to next available object. If getNext() returns NULL, it means we reached the last object. Client can use getKey, getKeyLength, getValue, and getValueLength to get object data of current object.

This class is used by client applications to make range queries using indexes. Each instance represents a single range query. To start a range query, a client creates an instance of this class. The client can then call getNext() to retrieve the objects in the desired range. For each object returned by getNext(), the caller can invoke getKey(), getKeyLength(), getValue(), and getValueLength() to get information about that object.

Red Flag: Implementation contaminates interface documentation

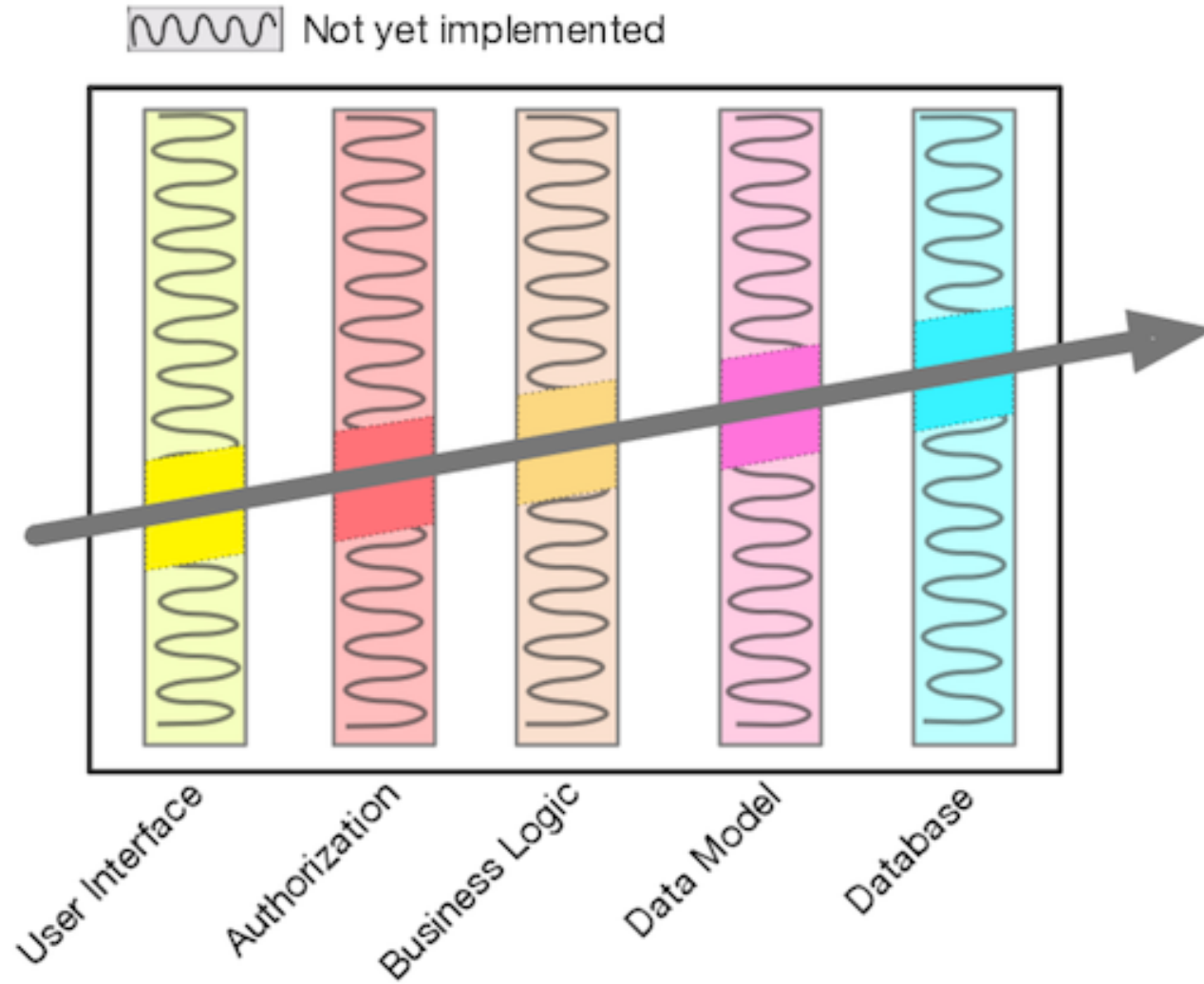
This red flag occurs when interface documentation, such as that for a method, describes implementation details that aren't needed in order to use the thing being documented.

Tips & Tools

Refactoring

- Altering the structure of the code without changing its behaviour
- Refactor when:
 - You notice duplication
 - You notice shallow modules
 - Your knowledge is outdated
- Refactor early, refactor often

Tracer bullets



Prototypes

- Disposable, incomplete code to try out an idea
- Prototyping is a learning experience
- You can ignore:
 - Correctness: use dummy data
 - Completeness
 - Robustness
 - Style
- **Agents are perfect for prototyping**

Don't outrun your headlights

- Take small steps
- Avoid future-telling:
 - Estimating completion dates far in the future
 - Guessing future needs
 - Guessing future results

External configuration

- Values that may change should go in config files
- Don't overdo it

Other tips & tools

- Become a power editor
 - **Challenge:** learn vim bindings
- Master the shell
 - **Challenge:** live in the terminal
- Read the damn error message
- Assertive programming

Testing

Unit testing

- Automated test of a specific module
- In R: `testthat`
- In Python: `unittest`/`pytest`
- I recommend using AI for tests

Unit testing

```
import unittest

class TestStringMethods(unittest.TestCase):

    def setUp(self):
        pass

    # Returns True if the string contains 4 a.
    def test_strings_a(self):
        self.assertEqual('a'*4, 'aaaa')

    # Returns True if the string is in upper case.
    def test_upper(self):
        self.assertEqual('foo'.upper(), 'F00')

    def test_isupper(self):
        self.assertTrue('F00'.isupper())
        self.assertFalse('Foo'.isupper())

    def test_strip(self):
        s = 'geeksforgeeks'
        self.assertEqual(s.strip('geek'), 'sforgeeks')

    # Returns true if the string splits and matches
    # the given output.
    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        with self.assertRaises(TypeError):
            s.split(2)

if __name__ == '__main__':
    unittest.main()
```

```
source("../..R/functions.R")

test_that("fizzbuzz of 1 is 1", {
  expect_equal(fizzbuzz(1), 1)
})

test_that("fizzbuzz of 2 is 2", {
  expect_equal(fizzbuzz(2), 2)
})

test_that("fizzbuzz of 3 is 'fizz'", {
  expect_equal(fizzbuzz(3), "fizz")
})
```

Unit testing

- Testing is not about finding bugs
 - `def get_avid_viewers(): ...`
 - `def get_avid_users(db): ...`
 - `def get_avid_users(db, qualifying_field_name): ...`

Test-Driven Development

- Decide on a small piece of functionality to add
- Write a test that will pass once it is implemented
- Run all tests, verify your new test is the only one that fails
- Write the smallest amount of code to get the test to pass
- Refactor
- **Don't overdo it**

Property-based testing

- Unit tests check specific examples
 - “Does `sort([3,1,2])` return `[1, 2, 3]`?”
- Property-based testing checks universal properties
 - “Does `sort(x)` always return a sorted list for any `x`?”
- In python: hypothesis
- In R: quickcheck

Property-based testing

```
def test_reverse_list():  
    assert reverse([1, 2, 3]) == [3, 2, 1]  
    assert reverse([]) == []  
    assert reverse([5]) == [5]
```

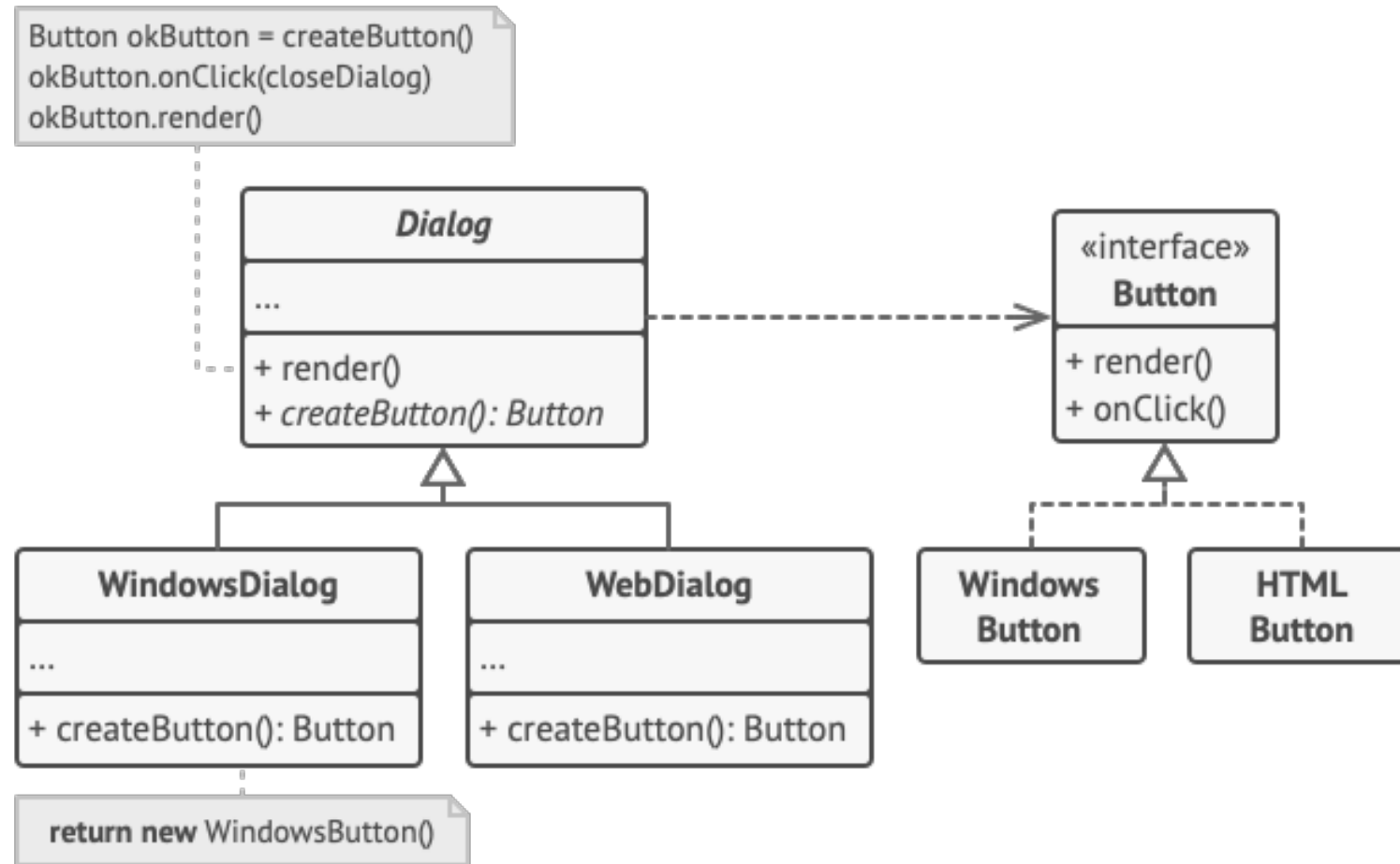
```
from hypothesis import given  
import hypothesis.strategies as st  
  
@given(st.lists(st.integers()))  
def test_reverse_inverts_twice(lst):  
    assert reverse(reverse(lst)) == lst
```

Design Patterns

Design patterns

- A **design pattern** is a typical solution to common problem in software design, like a blueprint
- Benefits:
 - Toolkit of tried and tested solutions
 - Define a common language
- Drawbacks:
 - Can be viewed as dogmatic
 - If all you have is a hammer, everything looks like a nail

Factory Method



Dependency Injection

```
class EmailProvider:
    def send_email(self, message: str, recipient: str):
        print(f"Sending Email to {recipient}: {message}")

class NotificationService:
    def __init__(self):
        self.email_provider = EmailProvider()

    def send_notification(self, message, recipient):
        self.email_provider.send_email(message, recipient)

if __name__ == "__main__":
    emailService = NotificationService()
    emailService.send_notification("Hello via Email!", "abc@example.com")
```

Dependency Injection

```
class NotificationProvider:
    def send_notification(self, message: str, recipient: str):
        raise NotImplementedError()

class EmailProvider(NotificationProvider):
    def send_notification(self, message: str, recipient: str):
        print(f"Sending Email to {recipient}: {message}")

class SMSProvider(NotificationProvider):
    def send_notification(self, message: str, recipient: str):
        print(f"Sending SMS to {recipient}: {message}")

class NotificationService:
    def __init__(self, notificationProvider: NotificationProvider):
        self.notificationProvider = notificationProvider

    def send_notification(self, message: str, recipient: str):
        self.notificationProvider.send_notification(message, recipient)

if __name__ == "__main__":
    email_provider = EmailProvider()
    email_service = NotificationService(email_provider)
    email_service.send_notification("Hello via Email!", "abc@example.com")

    sms_provider = SMSProvider()
    sms_service = NotificationService(sms_provider)
    sms_service.send_notification("Hello via SMS!", "123-456-7890")
```

Parameter Object

Customer
amountInvoicedIn (start : Date, end : Date) amountReceivedIn (start : Date, end : Date) amountOverdueIn (start : Date, end : Date)

Customer
amountInvoicedIn (date : DateRange) amountReceivedIn (date : DateRange) amountOverdueIn (date : DateRange)

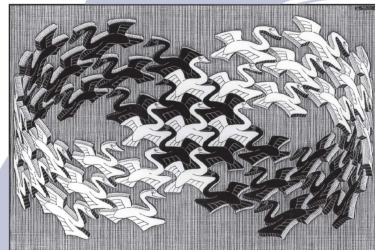
More on design patterns



Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch



ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES

nilbuild / [design-patterns-for-humans](#) Public [Sponsor](#) [Notifications](#)

[Code](#) [Issues](#) 5 [Pull requests](#) 11 [Actions](#) [Projects](#) [Security and](#)

[master](#) [Go to file](#) [Code](#)

[ebcq](#) Fix link in readme.md (#143) 7023f30 · 2 years ago

.github	Add banner	3 years ago
readme.md	Fix link in readme.md (#143)	2 years ago

README

for humans

DESIGN PATTERNS

🌈 Ultra-simplified explanation to design patterns! 🌈

A topic that can easily make anyone's mind wobble. Here I try to make them stick in to your mind (and maybe mine) by explaining them in the *simplest* way possible.